

■ Day 5 — Redshift Join Internals & Optimization

■ Topics Covered

- Hash Join
- Merge Join
- Nested Loop Join
- Broadcast Join
- Redistribution
- Local Joins
- WLM Concepts
- Query Optimization

■ customers Table

```
CREATE TABLE customers (  
    customer_id BIGINT,  
    customer_name VARCHAR(100),  
    city_id INT  
)  
DISTSTYLE KEY  
DISTKEY(customer_id)  
SORTKEY(customer_id);
```

■ transactions Table

```
CREATE TABLE transactions (  
    transaction_id BIGINT,  
    customer_id BIGINT,  
    merchant_id BIGINT,  
    amount DECIMAL(10,2),  
    transaction_time TIMESTAMP  
)  
DISTSTYLE KEY  
DISTKEY(customer_id)  
SORTKEY(customer_id);
```

■ merchants Table

```
CREATE TABLE merchants (  
    merchant_id BIGINT,  
    merchant_name VARCHAR(100),  
    category VARCHAR(50)  
)  
DISTSTYLE ALL;
```

■ cities Table

```
CREATE TABLE cities (  
    city_id INT,  
    city_name VARCHAR(50)  
)  
DISTSTYLE ALL;
```

■ Sample Data

```

INSERT INTO customers VALUES
(101, 'Rohan', 1),
(102, 'Amit', 2),
(103, 'Neha', 3);

INSERT INTO transactions VALUES
(5001, 101, 9001, 1200, '2026-07-01 10:00:00'),
(5002, 102, 9002, 2500, '2026-07-01 10:10:00');

INSERT INTO merchants VALUES
(9001, 'Amazon', 'Shopping'),
(9002, 'Swiggy', 'Food');

INSERT INTO cities VALUES
(1, 'Mumbai'),
(2, 'Pune');

```

■ HASH JOIN

Most common join in Redshift.

Internal Steps:

1. Smaller table loaded into memory
2. Hash table created
3. Large table probes hash table

Hash lookup complexity → $O(1)$

```

EXPLAIN
SELECT
    c.customer_name,
    t.amount
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id;

```

■ MERGE JOIN — FASTEST JOIN

Requirements:

- Same DISTKEY
- Same SORTKEY
- Sorted Data

Advantages:

- No hashing
- Low memory usage
- Sequential scan

```

EXPLAIN
SELECT
    c.customer_name,
    t.amount
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
ORDER BY c.customer_id;

```

■ NESTED LOOP JOIN

For every row another table scanned.

Complexity → $O(n^2)$

Very expensive for large datasets.

```
EXPLAIN
SELECT *
FROM transactions t
JOIN customers c
ON t.amount > c.customer_id;
```

■ BROADCAST JOIN

Small tables copied to all nodes.

DISTSTYLE ALL helps avoid moving large fact tables.

```
EXPLAIN
SELECT
    t.transaction_id,
    m.merchant_name
FROM transactions t
JOIN merchants m
ON t.merchant_id=m.merchant_id;
```

■ REDISTRIBUTION

Rows move across nodes during joins.

Causes:

- network overhead
- slow joins
- memory pressure

```
CREATE TABLE bad_transactions (
    transaction_id BIGINT,
    customer_id BIGINT,
    status VARCHAR(20),
    amount DECIMAL(10,2)
)
DISTSTYLE KEY
DISTKEY(status);
```

■ Best Optimized Join

- Merge Join → Fastest
- Hash Join → Most practical
- Nested Loop → Worst

Hash joins dominate real production because most data is unsorted.

■ WLM (Workload Management)

Queues created for:

- ETL
- Dashboards
- Analysts
- ML workloads

Helps isolate workloads and manage concurrency.

■ Day 5 Outcome

- Learned Join Internals
- Practiced Hash Join

- Practiced Merge Join
- Understood Nested Loop Problems
- Learned Redistribution
- Learned Broadcast Join
- Learned WLM Concepts